EC4219: Software Engineering (Spring 2025)
Homework 1: Bounded Verification of Mutual Exclusion

Due: 6/6, 23:59 (submit on GIST LMS)

Instructor: Sunbeom So

Important Notes

- **Evaluation criteria**
  The correctness of your implementation will be evaluated using testcases:

$$\frac{\#\text{Passed}}{\#\text{Total}} \times 100$$

  - "Total": testcases prepared by the instructor (not disclosed before the evaluation).
  - "Passed": testcases where expected outputs match your outputs.

- **Compilable**
  Make sure that your submission is successfully compiled. If your code cannot be compiled, you will get 0 points for the corresponding HW.

- **Academic Integrity**
  Violating academic integrity will result in an F, even after the end of the semester.

  Click here to check the rules related to academic integrity.

  Be aware that proving your academic integrity is entirely your responsibility.

- **No Changes on Templates, File Names, and File Extensions**
  Your job is to complete `(* TODO *)` parts in provided code templates. You should not modify the other templates. Do not change the file names. The submitted files should have `.ml` extensions, not the others (e.g., `.pdf`, `.zip`, `.tar`).

- **Regarding Using LLM-based Tools**
  You can use LLM-based tools to complete your HW, as long as (1) you use LLMs by yourself, (2) you can reproduce all the steps assisted by LLMs, and (3) you can clearly explain all the details of your implementation (including LLM-generated parts).

  Even though you claim that you have completed HW with the help of LLMs, if your submission is highly similar to other students' code and you cannot reproduce certain steps by any reason (including the nondeterminism of LLMs), I will consider that you committed academic misconduct.

1 Goal

Your goal is to implement a verifier to prove or disprove mutual exclusion safety within a bounded number of execution steps.

Before starting this assignment, please make sure that you have installed OCaml and Z3. Click this link for the detailed instructions.

2 Input-Output Example

Consider the example program from `hw1/benchmark/unsafe1`:

```
--- Thread_A ---
(0, maygoto 1)
(1, if p then goto 1 else goto 2)
(2, p=true and goto 3)
(3, critical and goto 4)
(4, p=false and goto 0)

--- Thread_B ---
(0, maygoto 1)
(1, if p then goto 1 else goto 2)
(2, p=true and goto 3)
(3, critical and goto 4)
(4, p=false and goto 0)
```

The program consists of the two threads, and each thread can be in one of the five states (0–4). The meaning of each command is self-explanatory. We assume that all shared variables (e.g., `p`) are initially set to *false*.

Your goal is to prove or disprove whether mutual exclusion is guaranteed within a preset bound on the execution steps, i.e., whether the two threads cannot enter the critical section (`critical`) simultaneously. For example, if you run the command

```
$ ./main.native -input ../benchmark/unsafe1 -max_bound 3
```

your verifier should return the following result:

```
[INFO] current bound: 1
[INFO] current bound: 2
[INFO] current bound: 3
===== Verification Result ======
Safe
Time: ...
```

However, using the command with the increased verification bound:

```
$ ./main.native -input ../benchmark/unsafe1 -max_bound 50
```

you should obtain the following result:

```
[INFO] current bound: 1
[INFO] current bound: 2
[INFO] current bound: 3
[INFO] current bound: 4
[INFO] current bound: 5
[INFO] current bound: 6
===== Verification Result ======
Unsafe at bound 6
Error trace: (0,0) -> (1,0) -> (1,1) -> (2,1) -> (2,2) -> (3,2) -> (3,3)
Time: ...
```

which states that mutual exclusion can be violated at step 6.

3 Structure of the Project

You can find the following files in the directory `hw1/code`.

- `main.ml`: contains the driver code that invokes the main logic (the function `run` from `verify.ml`).
- `pgm.ml`: implements the abstract syntax of programs, which is defined below:

$$\begin{array}{lcl} p & \rightarrow & (\tau_A, \tau_B) \\ h & \rightarrow & (tid, lcmd^*) \\ lcmd & \rightarrow & (l, c) \\ c & \rightarrow & \mathsf{maygoto}\ l \mid \mathsf{if}\ (p, l, l') \mid \mathsf{set}(v, b, l) \mid \mathsf{critical}(l) \end{array}$$

  A program p to verify consists of two threads (τ_A, τ_B). A thread τ is a pair of its identifier ($tid \in \{A, B\}$) and a sequence ($lcmd^*$) of labeled commands. In a labeled command (l, c), l denotes the label for the current state of a thread, and c is the command to execute. The meaning of each command c is the following.

  - $\mathsf{maygoto}\ l'$: A thread can either stay in the current state or move to another state l'.
  - $\mathsf{if}\ (p, l, l')$: If the value of the shared variable p is *true*, a thread goes to the state l. Otherwise, the thread goes to the state l'.
  - $\mathsf{set}(v, b, l)$: A thread sets the shared variable v to a truth value $b \in \{\mathsf{true}, \mathsf{false}\}$, and then goes to the state l.
  - $\mathsf{critical}(l)$: A thread enters the critical section, and then goes to the state l.

  For example, using the grammar above, the program p in `hw1/benchmark/unsafe1` can be expressed as:

$$\begin{array}{l} p = (\tau_A, \tau_B), \quad \tau_A = (A, lcmds_A), \quad \tau_B = (B, lcmds_B), \\ lcmds_A = (0, \mathsf{maygoto}\ 1) \cdot (1, \mathsf{if}(p, 1, 2)) \cdot (2, \mathsf{set}(p, \mathsf{true}, 3)) \cdot (3, \mathsf{critical}(4)) \cdot (4, \mathsf{set}(p, \mathsf{false}, 0)) \\ lcmds_B = (0, \mathsf{maygoto}\ 1) \cdot (1, \mathsf{if}(p, 1, 2)) \cdot (2, \mathsf{set}(p, \mathsf{true}, 3)) \cdot (3, \mathsf{critical}(4)) \cdot (4, \mathsf{set}(p, \mathsf{false}, 0)) \end{array}$$

Algorithm 1 The function `run` in `verify.ml`

Input: A program p to verify
Input: The maximum verification bound k_{max} ▷ `!Options.max_bound` in the code
Output: Verification result ▷ $(k, model)$:

1: $k_{cur} \leftarrow 1$ ▷ current verification bound (`cur_bound` in the code)
2: **while** *true* **do**
3: **if** $k_{cur} > k_{max}$ **then**
4: **return** $\perp$ ▷ p ensures mutual exclusion in k_{max} steps
5: $\phi \leftarrow \text{GenVc}(p, k_{cur})$ ▷ $\phi : \phi_{init} \wedge \phi_{one} \wedge \phi_{pgm} \wedge \neg\phi_{safe}$
6: $result, model \leftarrow \text{Z3-Solver}(\phi)$
7: **if** $result = \mathsf{SAT}$ **then** ▷ ϕ_{safe} can be violated (i.e., $\neg\phi_{safe}$ can be true)
8: **return** $(k_{cur}, model)$ ▷ mutual exclusion can be violated at time k_{cur} under $model$
9: **else if** $result = \mathsf{UNSAT}$ **then**
10: $k_{cur} \leftarrow k_{cur} + 1$

- `verify.ml`: should implement the verification logic to prove or disprove mutual exclusion safety. **Your job is to complete and submit this file only. Please do not modify the file name.** In the file, the function `run` implements the top-level verification routine defined in Algorithm 1. Feel free to implement any additional helper functions, even if they are not specified in this file.

  Your main job is to complete the function `gen_vc` (GenVC in Algorithm 1) that generates the following verification condition ϕ:

$$\phi \iff \phi_{init} \wedge \phi_{one} \wedge \phi_{pgm} \wedge \neg\phi_{safe}$$

  – ϕ_{init} indicates the initial constraint that must be satisfied at step 0. Specifically, it enforces: (1) the two threads τ_A and τ_B should be in state 0 rather than any other states, and (2) all shared variables (Z) are initially set to *false*.

$$\phi_{init} \iff A_{0,0} \wedge B_{0,0} \wedge \bigwedge_{i \in [1, \mathsf{last}_A]} \neg A_{i,0} \wedge \bigwedge_{i \in [1, \mathsf{last}_B]} \neg B_{i,0} \wedge \bigwedge_{z \in Z} \neg z_0$$

  Here, a propositional variable $A_{l,t}$ (resp., $B_{l,t}$) expresses that a thread τ_A (resp., τ_B) is in state l at time t. last_A and last_B are the last state labels of τ_A and τ_B respectively. z_t is a propositional variable for denoting that a shared variable $z \in Z$ is set to *true* at time t.

  – The constraint ϕ_{one} expresses that a thread can be in exactly one state at a time, i.e., a thread cannot be in two states simultaneously.

$$\phi_{one} \iff \bigwedge_{t \in [1, k_{cur}]} \bigwedge_{\substack{x \neq y, \\ x,y \in [1, \mathsf{last}_A]}} \neg(A_{x,t} \wedge A_{y,t}) \wedge \bigwedge_{t \in [1, k_{cur}]} \bigwedge_{\substack{x \neq y, \\ x,y \in [1, \mathsf{last}_B]}} \neg(B_{x,t} \wedge B_{y,t})$$

  Here, k_{cur} is the current verification bound on the number of execution steps.

  – The constraint ϕ_{pgm} encodes the execution semantics of a program.

$$\begin{aligned} \phi_{pgm} \iff & \bigwedge_{t \in [0, k_{cur}-1]} \bigwedge_{(l,c) \in lcs_A} \mathsf{encode}(A, (l, c), t, Z, E_t) \\ & \wedge \bigwedge_{t \in [0, k_{cur}-1]} \bigwedge_{(l,c) \in lcs_B} \mathsf{encode}(B, (l, c), t, Z, \neg E_t) \end{aligned}$$

In the above, lcs_A and lcs_B denote the sets of labeled commands in threads τ_A and τ_B, respectively:

$$lcs_A = \{(l_i, c_i) \mid A = (-, lcmds), lcmds = (l_0, c_0) \cdots (l_k, c_k), 0 \leq i \leq k\},$$
$$lcs_B = \{(l_i, c_i) \mid B = (-, lcmds), lcmds = (l_0, c_0) \cdots (l_k, c_k), 0 \leq i \leq k\}$$

Z denotes the set of shared variables in the program. E_t is a propositional variable that determines which thread is executed at time t; thread τ_A is executed iff $E_t \iff true$, and thread τ_B is executed iff $E_t \iff false$. encode is defined as follows:

$$\mathsf{encode}(H, (l, c), t, Z, G) \iff$$
$$\begin{cases} (\neg G \wedge H_{l,t} \to H_{l,t+1}) \wedge (G \wedge H_{l,t} \to (H_{l,t+1} \vee H_{l',t+1}) \wedge \psi) & \text{if } c = \mathsf{maygoto}\ l' \\ (\neg G \wedge H_{l,t} \to H_{l,t+1}) \wedge & \text{if } c = \mathsf{if}\ (p, l', l'') \\ (G \wedge H_{l,t} \wedge p \to H_{l',t+1} \wedge \psi) \wedge (G \wedge H_{l,t} \wedge \neg p \to H_{l'',t+1} \wedge \psi) & \\ (\neg G \wedge H_{l,t} \to H_{l,t+1}) \wedge (G \wedge H_{l,t} \to H_{l',t+1} \wedge v_{t+1} \wedge \psi') & \text{if } c = \mathsf{set}\ (v, \mathsf{true}, l') \\ (\neg G \wedge H_{l,t} \to H_{l,t+1}) \wedge (G \wedge H_{l,t} \to H_{l',t+1} \wedge \neg v_{t+1} \wedge \psi') & \text{if } c = \mathsf{set}\ (v, \mathsf{false}, l') \\ (\neg G \wedge H_{l,t} \to H_{l,t+1}) \wedge (G \wedge H_{l,t} \to H_{l',t+1} \wedge \psi) & \text{if } c = \mathsf{critical}\ (l') \end{cases}$$

where the constraint ψ (resp., ψ') enforces that the values of shared variables in Z (resp., $Z' = Z \setminus \{v\}$) remain unchanged:

$$\psi \iff \bigwedge_{z \in Z} z_t \leftrightarrow z_{t+1}, \qquad \psi' \iff \bigwedge_{z \in Z'} z_t \leftrightarrow z_{t+1}$$

– The constraint ϕ_{safe} encodes the mutual exclusion condition, i.e., the two threads are not in the critical section simultaneously at the final step (k_{cur}).

$$\phi_{safe} \iff \neg(\bigvee_{l \in \mathsf{Critical}_A} A_{l,k_{cur}} \wedge \bigvee_{l \in \mathsf{Critical}_B} B_{l,k_{cur}})$$

Here, $\mathsf{Critical}_A$ (resp., $\mathsf{Critical}_B$) indicates the set of state labels for the critical section in thread τ_A (resp., τ_B).

- `verify_utils.ml`: provides the helper functions to generate propositional variables.
- `solver.ml`: implements the interface to invoke the Z3 SMT solver.
- `formula.ml`: implements the definition of the propositional logic formulas.
- `trace.ml`: implements the functionality to print an error trace, if a given program contains mutual exclusion errors.
- `*.mli` files are the interfaces for the modules implemented in `*.ml` files. Only definitions declared in `*.mli` can be accessed from external modules.
- `parser.mly` and `lexer.mll`: contain the lexer and parser specifications in `ocamllex` and `ocamlyacc`, respectively. You do not need to understand these details for this assignment.

4 How to Build

Make the build script executable by running the command below in the directory `hw1/code`:

```
$ chmod +x build
```

Once you complete `verifier.ml`, you can build the project:

```
$ ./build
```

Then, the executable `./main.native` will be generated. You can run it as follows.

```
$ ./main.native -input YOUR_TESTCODE -max_bound YOUR_BOUND
```